

1. Gestion du code source

- **Problème : déclencheurs de pipeline manquants ou mal configurés.**

Symptômes : après un commit/push, aucun build n'est lancé (ou le pipeline reste en veille), sans erreur évidente dans l'interface CI. *Causes* : Webhooks ou triggers non configurés, filtrage de branche erroné, annotation de commit pour sauter le CI.

Diagnostics : vérifier les logs de webhook (GitHub/GitLab) et les paramètres de projet, s'assurer que le fichier de pipeline (Jenkinsfile, .gitlab-ci.yml, .github/workflows/*.yml) est présent et valide. Par exemple, Jenkins exige que les triggers soient configurés dans le Jenkinsfile (par ex. triggers { githubPush() } dans un pipeline déclaratif). GitLab CI refuse de lancer un pipeline si le YAML est invalide – un badge “yaml invalid” apparaît. Utiliser l'outil **CI Lint** de GitLab pour valider le YAML. GitHub Actions ignore les commits contenant [ci skip] ou [skip ci] et peut ne déclencher que sur la branche par défaut. Azure DevOps peut avoir les déclencheurs désactivés dans l'UI (“Disable YAML CI trigger”) ou des filtres include/exclude mal réglés. CircleCI exige que le projet soit actif et que les filtres de tags/branches autorisent le build (par ex. un push de tag sans filtre renvoie “No workflow”).

Solutions : Mettre à jour la configuration de déclenchement : pour Jenkins, ajouter dans Jenkinsfile le bloc triggers { githubPush() }; pour GitLab/GitHub/Azure, corriger les YAML de trigger (on: push/branch ou triggers: branches), retirer les [ci skip], et ajuster les filtres de branche. S'assurer que les webhooks Git sont actifs et que l'accès (token, clé SSH) est valide. En cas de problème de branche renommée (ex. “master” en “main”), mettre à jour la configuration. *Outils spécifiques* : voir les docs GitLab sur le lint et les variables CI, la doc GitHub sur les filtres de chemin et commits skip, et les options de pipeline Azure.

2. Phase de compilation (Build)

- **Environnement mal défini / dépendances manquantes.**

Symptômes : erreurs “command not found”, bibliothèques absentes, versions incompatibles. *Causes* : divergence entre l'environnement local et l'environnement du pipeline. Par exemple, certains outils sont installés localement mais pas dans l'image Docker ou l'agent CI, ou les versions de langages (Node, Python, Java) diffèrent. *Diagnostics* : examiner le journal de build et lister les versions installées (node -v, python --version, etc.), vérifier le contenu des conteneurs ou agents (dpkg -l, apk info).

Solutions : Utiliser un environnement d'exécution explicite (image Docker ou machine virtuel) avec les versions requises. Par exemple, verrouiller les dépendances avec un package-lock.json ou équivalent et utiliser des conteneurs

reproductibles. Installer explicitement les outils nécessaires (par ex. apt-get install, sdk install java, nvm use, etc.). Pour Jenkins, en cas d'**OutOfMemoryError**, augmenter la mémoire allouée au jvm (-Xmx) ou au conteneur d'agent. Nettoyer les caches ou artefacts temporaires pour libérer de l'espace (commande docker system prune, augmenter quotas disque). *Outils spécifiques* : sur Jenkins, configurer les labels d'agent pour correspondre à l'environnement voulu ; sur CircleCI/GitLab, vérifier le Dockerfile ou la machine de build.

- **Gestion des dépendances.**

Symptômes : build cassé à cause de versions inattendues ou de paquets récents introduisant des bogues. *Causes* : dépendances non verrouillées ou non mises à jour. *Diagnostics* : comparer les versions résolues avec le lockfile, analyser les logs d'erreur ("ModuleNotFoundError", "package not found"). *Solutions* : Pinner les versions (ex. pip freeze > requirements.txt, npm ci). Mettre à jour régulièrement (CI Lint dépendances), utiliser des outils comme Dependabot. *Conseil* : comme dans SquareOps, garder les fichiers de lock à jour prévient ce type d'erreur.

- **Timeouts / Ressources insuffisantes.**

Symptômes : builds qui plantent par dépassement de temps ou manquent de CPU/RAM. Par exemple, Jenkins montre java.lang.OutOfMemoryError: GC overhead limit exceeded, ou un job Docker est tué (OOMKilled). *Causes* : quotas mal ajustés ou tests trop longs. *Diagnostics* : voir les logs de ressources (free -h, docker stats). *Solutions* : augmenter le timeout dans la config de pipeline, allouer plus de mémoire au conteneur/agent (par ex. resources.limits.memory en GitLab), paralléliser les tests. Diviser les étapes ou utiliser le cache CI pour accélérer (ex. cache de modules Python/Node). *Outils* : Jenkins Pipeline a des directives timeout ; Kubernetes pods utilisés par Tekton ou ArgoCD peuvent nécessiter resources.requests/limits.

- **Cache et artefacts de build obsolètes.**

Symptômes : build utilise d'anciennes dépendances ou fichiers. *Solutions* : forcer la réinitialisation du cache (dans CircleCI/GitLab CI, invalidation du cache), ou automatiser clean (par exemple, mvn clean, gradle --refresh-dependencies).

- **Outils de compilation manquants.**

Symptômes : erreurs comme "command not found: mvn" ou "npm: command not found". *Solutions* : Installer les outils dans l'image de build ou l'agent (ex. yum install maven ou utiliser l'image openjdk:latest pour Maven).

3. Phase de test

- **Tests instables (flaky) ou environnement de test mal configuré.**

Symptômes : échecs de tests intermittents sans modifications de code, build parfois vert/sometimes rouge. *Causes* : tests non déterministes, dépendances externes non simulées, partage d'état entre tests. Les tests dépendants de réseau ou de temporisation peuvent échouer de façon aléatoire. *Diagnostics* : exécuter les tests en local plusieurs fois, vérifier les logs pour voir la variabilité. Par exemple, environ 13% des échecs de tests sont dus à de la flakiness. *Solutions* : isoler les tests (mockier les API réseau), augmenter la tolérance aux temporisations (ex. ajuster les wait), réessayer automatiquement les tests connus instables. Utiliser des bibliothèques de « retry », ou exécuter plusieurs fois pour confirmer l'échec. S'assurer que l'environnement de test est complet (par ex. services en conteneurs via Docker Compose ou services stubs). *Bonnes pratiques* : exécuter les tests dans un environnement reproductible (Docker, conteneurs) pour réduire la variance, et éviter les dépendances mutuelles (la recherche montre que ~15% des tests flakies viennent d'un état partagé).

- **Tests manquants ou mal détectés.**

Symptômes : suite de tests passe sans rien exécuter (problème de détection des tests). *Causes* : noms de fichiers ou de méthodes mal nommés, pattern de test incorrect dans le pipeline. *Solution* : vérifier la configuration de l'étape test (par ex. le path des tests dans GitLab CI ou le pattern `**/*Tests.java` dans JUnit).

4. Gestion des artefacts

- **Artefact introuvable après build.**

Symptômes : étapes de déploiement échouent ("artefact not found"). *Causes* : chemin d'artefact erroné ou suppression prématurée. *Diagnostics* : vérifier le workspace du job pour voir si le fichier est produit, contrôler les paramètres d'archivage (ex. Jenkins archiveArtifacts, GitLab artifacts:paths). *Solutions* : corriger les chemins, conserver les artefacts (ex. artifacts: expire_in plus long). S'assurer qu'une étape de déploiement utilise bien le workspace du build ou télécharge l'artefact du stockage CI.

- **Échec de publication vers un repository d'artefacts (Artifactory, Nexus, registry Docker).**

Symptômes : erreurs "403 Forbidden", "Unauthenticated" ou "No such host" lors du push/publish. *Causes* : mauvaises variables d'authentification, URL mal configurée, token expiré. *Diagnostics* : messages d'erreur précis ("Permission denied"), tester manuellement les commandes (curl sur l'endpoint). *Solutions* : vérifier et régénérer les tokens/credentials (par ex. docker login, configurer les variables d'environnement CI), valider l'URL du repository. Pour Docker, s'assurer que imagePullSecrets (Kubernetes) ou la connexion docker login est

correcte. *Outils spécifiques* : Jenkins Artifactory plugin (configurer Server ID et spec), GitLab CI Registry (variable \$CI_JOB_TOKEN), Azure Artifacts (Service Connection).

- **Conflits de version d'artefact.**

Symptômes : versions écrasées ou pipeline en boucle (ex. pushing toujours le même snapshot). *Causes* : mauvaise incrémentation de version, usages de latest en Docker. *Solutions* : utiliser des tags uniques (par ex. hash du commit), ne pas écraser le repository, employez des versionnements sémantiques.

5. Phase de déploiement

- **Manifestes ou charts invalides.**

Symptômes : déploiement échoue à cause d'erreurs de syntaxe YAML/Helm. Par exemple, Helm affiche "parse error" sur un Chart{{metadata}}. *Diagnostics* : exécuter helm lint ou helm template --debug pour valider le chart. La démo OneUptime montre qu'une erreur de parenthèse dans le template Helm produit « Error: parse error ». *Solutions* : corriger les fichiers (helm template monchart . aide au debug).

- **CrashLoopBackOff (Kubernetes).**

Symptômes : pod en boucle de crash. *Causes* : application qui plante à l'exécution, dépendance manquante, arguments incorrects, manque de ressources. *Diagnostics* : utiliser kubectl logs <pod> pour voir l'erreur sous-jacente et kubectl describe pod <pod> pour les événements. *Solutions* : corriger l'application (par ex. installer la dépendance manquante) ou ajuster les ressources. L'erreur "CrashLoopBackOff" signifie simplement qu'il y a une erreur non liée au scheduler. Après correction, relancer le déploiement (ou kubectl rollout restart).

- **ImagePullBackOff / ErrImagePull (Kubernetes).**

Symptômes : pods indéfiniment en attente de téléchargement d'image. *Causes* : nom d'image ou tag incorrect, problème d'authentification au registry, réseau. *Diagnostics* : kubectl describe pod montre l'événement "ImagePullBackOff" ou "ErrImagePull". Vérifier les logs du cluster pour des erreurs d'accès. *Solutions* : corriger l'image dans le manifeste et configurer l'authentification (secret docker). Vérifier que l'image existe sur le registry. Tester la connectivité (kubectl get secret pour Docker registry, curl réseau vers le registry).

- **NodeNotReady (Kubernetes).**

Symptômes : nœud indisponible pour exécuter les pods. *Diagnostics* : kubectl get nodes, puis kubectl describe node <nœud>.

- **Fail Deployment (Kubernetes).**

Symptômes : kubectl rollout status deployment/xxx reste bloqué ou timeout (Spinnaker “Wait for manifest to stabilize” dépassement). *Cause notable* : pas assez de ressources pour monter les pods (CPU/Mémoire).

Solutions : augmenter quotas ou détruire pods bloqués, vérifier la configuration du scheduler (taints/affinity). Sur Spinnaker, consulter les logs clouddriver pour les erreurs d’infrastructure.

- **Helm Upgrade échoué après installation précédente.**

Symptômes : helm upgrade return code != 0, Release en état FAILED. *Solutions* : faire helm rollback au besoin. Désinstaller le release et réinstaller si nécessaire (helm uninstall). Utiliser helm history pour diagnostiquer.

- **ArgoCD sync / Tekton trigger.**

Symptômes : ArgoCD reste “OutOfSync” ou pipelines Tekton non déclenchés. *Causes* : permissions Kubernetes (RBAC), cluster non accessible, Webhook Tekton mal configuré. *Diagnostics* : vérifier les logs de l’Application Argo (argocd app logs), utiliser tkn eventlistener logs pour Tekton Triggers. Activer le logging debug avec ConfigMap pour Tekton Triggers. *Solutions* : ajuster les droits (ServiceAccount ArgoCD, RBAC Tekton), re-déployer l’EventListener.

6. Infrastructure / Provisioning

- **Lockage d’état Terraform / Provider manquant.**

Symptômes : échec terraform apply avec une erreur de lock ou de provider. Message fréquent : ConditionalCheckFailedException: error acquiring state lock ou The state lock is in place. *Diagnostics* : lire l’erreur (“error acquiring state lock”) qui indique souvent un lock persistant d’un run précédent.

Solutions à court terme : retirer manuellement le lock avec terraform force-unlock <LOCK_ID>, ou attendre la fin de la process.

Solutions long terme : utiliser des backends d’état partagés (S3, Consul) avec verrouillage, éviter les modifications concurrentes.

- **Crédentials absents ou région non spécifiée.**

Symptômes : Terraform renvoie “NoCredentialProviders” ou “Provider not found”. *Solutions* : exporter les variables d’environnement (AWS_ACCESS_KEY_ID etc), vérifier provider.tf (région, project).

- **Limites de quotas cloud (API rate-limit, ressources épuisées).**

Symptômes : erreur 429 ou “Quota Exceeded”. *Solutions* : demander un quota plus élevé, ajouter des sleep ou retry dans le script Terraform.

- **Mises à jour d’infra incomplètes.**

Solutions : toujours passer par terraform plan pour inspecter avant apply, valider le plan. En cas de divergence, re-importer les ressources (terraform import).

- **Provisionnement de cluster K8s échoue.**

Symptômes : cluster unreachable, nœuds errants. *Solutions* : examiner les logs du service cloud, vérifier la santé des nœuds (kubectl get nodes), explorer les cloud controller manager logs.

7. Gestion de configuration (Ansible)

- **Inventaire mal configuré.**

Symptômes : ansible-playbook répond “ERROR! Provided hosts list is empty” ou “No hosts matched”. *Cause typique* : Ansible ne connaît pas le fichier d’inventaire. Il faut passer -i hosts.ini ou ajouter les hôtes dans /etc/ansible/hosts. *Solutions* : Utiliser ansible-playbook -i <inventory> ou créer correctement le fichier d’inventaire (ini/yaml).

- **Problème de connexion SSH (UNREACHABLE).**

Symptômes : tâches Ansible reportent “UNREACHABLE” sur certains hôtes. *Causes* : clés SSH manquantes ou mauvais utilisateur, hôte injoignable (firewall). *Diagnostics* : ansible all -m ping -vvv pour détails. *Solutions* : configurer --private-key, vérifier ansible_user, tester manuellement ssh.

- **Syntaxe YAML invalide.**

Symptômes : “ERROR! Syntax Error while loading YAML.” *Solutions* : valider le playbook via ansible-playbook --syntax-check, corriger l’indentation.

- **Module ou variable manquante.**

Symptômes : “module not found in ansible.” ou “undefined variable”. *Solutions* : installer le module (pip install ansible.module_name ou role via Ansible Galaxy), définir toutes les variables (ou fournir des default).

- **Permissions root (become) insuffisantes.**

Symptômes : “Permission denied” lors de l’exécution de tâches privilégiées. *Solutions* : ajouter become: yes, vérifier que l’utilisateur a sudo sans mot de passe ou fournir le mot de passe sudo.

8. Surveillance et observabilité

- **Absence de logs ou métriques.**

Symptômes : on ne voit pas les messages de build/test en temps réel ou les métriques de pipeline. *Causes* : pipeline non configuré pour transmettre les logs (ex. pas de volume partagé), exporter() mal configuré. *Solutions* : rediriger les sorties vers un agrégateur (par ex. envoyer les logs Jenkins vers ELK ou Loki). Mettre en place Prometheus pour les métriques de pipeline (durées, statut), Grafana pour les dashboards.

- **Alertes non déclenchées.**

Symptômes : un pipeline échoue mais pas d’alerte envoyée (Slack,

mail). *Causes* : hooks mal paramétrés. *Solutions* : configurer des étapes “post-failure” dans les pipelines (ex. Slack webhook si `$CI_PIPELINE_STATUS == "failed"`).

- **Rechercher des erreurs dans les logs.**

Technique : utiliser des requêtes ou regex pour filtrer les logs. Par exemple, FreeCodeCamp suggère de sauvegarder des recherches («Saved Searches») dans Kibana. On peut créer des filtres sur des champs (outil CI ou étape spécifique) ou du texte :

ex. error_message:"*timeout*" ou pipeline.logs.message:"build failed".

- **Détection de motifs d'erreur (regex).**

Exemples de motifs communs pour extraire les erreurs dans les logs :

- **Échecs de test** : rechercher FAILURE ou Error (regex .*FAILURE.* comme motif générique).
- **Memory/CPU** : OOMKilled, Killed, OutOfMemoryError. Regex (insensible à la casse) : /(?!i)\b(OutOfMemory|OOMKilled|Killed)\b/.
- **Accès refusé** : Permission denied|Unauthorized. Regex : /permission\sdenied|unauthorized/i.
- **Fichiers manquants** : No such file or directory. Regex : /no\s\such\sfile\b/i.
- **Timeout** : Timeout|timed out|Connection refused. Regex : /timeout|timed\sout/i.
- **Kubernetes** : CrashLoopBackOff, ImagePullBackOff. Regex : /CrashLoopBackOff|ImagePullBackOff/.
- **HTTP 404/500** : HTTP 404|HTTP 500. Regex : /HTTP\s(404|500)/.

Ces motifs permettent d'agréger et de filtrer automatiquement les logs (via `grep -i -E`, Kibana/LogQL, etc.) pour repérer rapidement les erreurs critiques.

9. Sécurité DevOps

- **Fuites ou expos secrets.**

Symptômes : clé AWS/GCP affichée dans les logs, ou rejection par scanner SAST (ex. Gitleaks détecte un token). **Solutions** : utiliser des magasins de secrets (Vault, Azure Key Vault) plutôt que des variables en clair. Mettre en place des scans de secrets (ex. actions GitHub gitleaks-action). En cas d'alerte, révoquer les clés exposées et refaire le déploiement.

- **Analyses SAST/DAST échouées.**

Symptômes : étapes de sécurité (Checkmarx, Snyk) font échouer la

pipeline. *Solutions* : corriger les vulnérabilités signalées ou, si fausse alerte, ajuster la configuration du scanner. Intégrer une règle pour permettre les builds en allowlist de dépendances spécifiques.

- **Certificats SSL ou permissions.**

Symptômes : erreurs de type “certificate verify failed” ou refus d’accès aux registres. *Solutions* : importer le certificat racine dans le store de l’agent, ou passer –insecure temporairement. Vérifier les droits IAM liés aux credentials pipeline.

10. Rollback / Reprise après incident

- **Rétablir une version antérieure.**

Kubernetes (Helm) : utiliser kubectl rollout undo deployment/<nom> ou helm rollback <release> <revision>.

Terraform : si la dernière terraform apply a corrompu l’infra, on peut revenir au dernier état stable (via sauvegarde state).

Pipeline : relancer le dernier build success ou taguer une ancienne release.

Bonnes pratiques : mettre en place du déploiement blue/green ou canary pour minimiser l’impact. Garder un plan de sauvegarde des bases de données avant migrations, tester le rollback en staging.

11. Orchestration de pipeline

- **Jobs bloqués / ressources manquantes.**

Symptômes : files d’attente de jobs qui n’avancent pas. *Causes* : plus de jobs que d’exécuteurs disponibles. *Solutions* : augmenter le nombre d’agents Jenkins ou de runners GitLab, optimiser l’affectation (labels Jenkins, tags GitLab), découper le pipeline en jobs parallèles.

- **Jobs manuels / stades d’attente.**

Symptômes : pipeline en pause sur un job “Manual” (GitLab) ou “Input” (Jenkins) ou étape d’approbation (Spinnaker). *Solutions* : vérifier s’il existe un stade de validation manuelle et le valider, ou repasser en automatique selon le besoin.

- **Boucles infinies ou dépendances circulaires.**

Symptômes : pipeline qui se déclenche sans fin. *Diagnostics* : analyser la configuration de triggering. *Solutions* : ajouter un tag [ci skip] sur les commits de déclenchement, ou ajuster la condition only/except (GitLab) ou on: [push] branches: [...] (GitHub) pour éviter les récursions.

- **Conflits de verrou (Locks) Jenkins.**

Symptômes : étapes qui ne démarrent pas car un verrou est détenu par une autre branche. *Solutions* : utiliser le plugin *Lockable Resources* avec parcimonie, ou augmenter les ressources Jenkins.

Problèmes dans les pipelines CI/CD

- **Syntaxe YAML ou Pipeline as Code invalide** – Erreur lors du parsing du fichier (GitLab CI, GitHub Actions, etc.). Causes : indentations incorrectes, variables mal citées. *Dépannage* : utiliser les validateurs intégrés (ex. *GitLab CI Lint*, *Azure YAML Validator*), lire attentivement les logs d’erreur de parsing. *Correction* : corriger la syntaxe ; préconiser des templates réutilisables et la validation lors de la revue de code.
- **Authentification au dépôt ou registry** – Échec de git clone ou docker push/pull (Jeton expiré, permissions manquantes). Causes : jeton non valide, rôle IAM absents, mauvais scope OAuth. *Dépannage* : vérifier les variables secrètes (p. ex. `CI_JOB_TOKEN`, `GITHUB_TOKEN`), s’assurer que l’utilisateur/compte de service a les droits requis (lecture repo, push registry). *Solution* : régénérer le token, ajouter les rôles “Project Viewer”, “Storage Object Admin” ou spécifiques (par ex. pour Cloud Build), et vérifier les scopes lors de l’attachement de SA. *Prévention* : automatiser le rafraîchissement des secrets et l’affectation correcte de rôles.
- **Runners/Agents indisponibles ou mal configurés** – Jobs en attente indéfiniment (« *Waiting for a runner/agent* »). Causes : le runner/agent est offline, pas assez de runners (concurrence atteinte), labels mal utilisés (GitHub Actions). *Dépannage* :
 - Vérifier l’état des runners/agents (UI GitLab/GitHub, `systemctl status gitlab-runner`, Azure DevOps agent pools, CircleCI Dashboard).
 - S’assurer des labels exclusifs pour GitHub self-hosted (éviter conflits de labels).
 - Tester la connectivité réseau et l’horloge système (pour GitHub runner décorrélé).
 - Consulter les logs du runner (`journalctl -u gitlab-runner` ou logs Azure).
 - Examiner la concurrence (CircleCI : runners « idle » mais jobs en queue). *Solution* : redémarrer ou réinstaller les runners, augmenter la capacité (ajouter des agents/hôtes) ou reconfigurer les labels/concurrence. *Bonnes pratiques* : surveiller l’agrégat de runners, utiliser des runners élastiques (Autoscaling, Cloud-hosted), configurer des timeout raisonnables et notifier les échecs vite (ex. intégrations Slack).
- **Cache et artefacts défaillants** – Cache non restauré (“cache miss”) ou artefacts introuvables. Causes : clé de cache mal formée, runners isolés sans stockage partagé, artefacts expirés ou stockés sur mauvais bucket. *Dépannage* :

- Pour le cache GitLab : vérifier la clé cache:key et le chemin (paths), et comprendre que chaque runner maintient son cache local (GitLab ne partage pas le cache entre runners).
- GitHub Actions : confirmer l'usage de la même key et la dépendance entre jobs.
- Azure/CircleCI : similarité, s'assurer que le chemin de cache existe au moment de la restauration. *Solution* : ajuster la stratégie de caching (clefs uniques, répertoires standard /tmp pour runners CircleCI), augmenter la limite de stockage si nécessaire, ou désactiver temporairement pour isoler.
Prévention : mettre en place des caches préchiffrés dans Artifact Registry ou utiliser des artefacts versionnés pour partage inter-pipelines.
- **Échecs de job** – Erreurs d'exécution, exit codes non nuls, tests qui échouent. *Causes* : bugs de script, dépendances manquantes, quotas dépassés, ressources insuffisantes, timeouts. *Dépannage* :
 - **Logs** : toujours examiner minutieusement les logs des jobs (CLI ou interface). Par exemple, Azure DevOps propose de passer en mode verbeux pour plus de détails. GitHub Actions permet d'activer le debug avec ACTIONS_STEP_DEBUG.
 - **Debug command** : reproduire la commande locale (npm install, msbuild, etc.) sur un agent ou machine équivalent.
 - **Timeout** : vérifier si la job a été tuée après délai (job time-out dans Azure, 60mn par défaut pour agents hébergés). Ajuster via paramètre timeoutInMinutes ou utiliser un agent self-hosted plus permissif.
 - **Quotas/Ressources** : si build Docker, assurer suffisamment de vCPU/mémoire (ex. Cloud Build : augmenter CPU). *Solution* : corriger la source de l'erreur (lib, commande manquante, path invalide). Ajouter des étapes de test locales et corriger l'environnement du runner (par ex. apt-get install avant npm install). *Prévention* : tests unitaires robustes, limiter les builds inutiles (branches non protégées), notifier dès le premier échec (fail-fast), surveiller quotas.
- **Tests instables (flaky tests)** – Tests qui passent aléatoirement, ruinent la confiance du pipeline. *Causes* : dépendances non isolées, environnement non déterministe, conflit de données, problèmes réseau intermittents. *Dépannage* : exécuter localement, identifier patterns, flakiness runner (ex : Chrome sans sandbox). Utiliser retries automatiques au besoin. *Solution* : isolation des tests

(env propres via Docker), mock des services externes, fixer les versions de dépendances (on évite le « drift » d'environnement). *Bonnes pratiques* : séparer tests lents/costly, paralléliser tests indépendants, linter de code. L'article Windframe évoque l'importance d'isoler les environnements et de réexécuter les tests stratégiquement.

- **Gestion des secrets** – Fuites ou échecs liés aux secrets (API keys, tokens). *Causes* : variables non définies, masquage manquant, limitations d'accès (ex. Secrets GitHub inaccessibles pour Pull Requests externes). *Dépannage* :
 - S'assurer que les secrets sont définis dans le projet (GitLab: Settings>CI/CD Variables, GitHub: repo/settings/secrets).
 - Vérifier les logs masqués (***) pour ne pas divulguer.
 - Pour GitHub, confirmer que l'action est dans un environnement correct (ex. seul le main ou workflow_call peut accéder aux secrets). *Solution* : accorder les scopes appropriés (ex. usage du GITHUB_TOKEN, autoriser l'accès aux secrets), ne jamais hard-coder les valeurs. *Prévention* : utiliser un gestionnaire de secrets dédié (HashiCorp Vault, Google Secret Manager intégré via Azure DevOps ou GitHub Actions), rotation régulière.
- **Dérive d'environnement** – Différence entre environnements (dev/staging/prod). *Causes* : configurations disparates (versions de bibliothèques, variables), déploiements manuels antérieurs. *Dépannage* :
 - Comparer les configurations (diff des fichiers IaC ou YAML).
 - Vérifier la version déployée correspond bien au commit/tag attendu. *Solution* : automatiser tout via infrastructure as code (Terraform, Helm charts), pipelines reproductibles, tester les déploiements dans un environnement miroir (CI-Dev). *Prévention* : maintien d'une unique source de vérité (ex. repo infra, gitops), revues de configuration.
- **Concurrence et limites de ressources** – Jobs mis en file d'attente ou tués pour manque de ressources. *Causes* : trop de jobs simultanés (limites CircleCI/Free tier), cluster saturé. *Dépannage* :
 - Vérifier la limite de concurrence dans le dashboard (CircleCI: [27†L591-L599]).
 - Contrôler la consommation de CPU/mémoire sur les agents (Jenkins nodes ou Kubernetes). *Solution* : limiter le nombre de builds par heure, augmenter les capacités (agents, runners), ou passer à un plan

supérieur. *Prévention* : prioriser les branches importantes, annuler les jobs redondants (Auto-cancel).

- **Intégrations tierces** – Intégrations avec des outils externes (Slack, Jira, dépôts privés, etc.) défaillantes. *Causes* : tokens expirés, API modifiée, webhook mal configuré. *Dépannage* : vérifier les logs d’erreurs dans les actions (les appels d’API échoués), tester les credentials. *Solution* : rafraîchir les tokens, vérifier les réglages webhooks (URL, certificats). *Prévention* : tenir à jour les intégrations selon la doc officielle (ex. Slack Orb de CircleCI, GitLab Slack Notifications).

Tableau : comparaison des problèmes CI/CD (gravité, fréquence, composants, temps moyen de résolution).

Problème	Gravité	Fréquence	Composant affecté	Temps de résolution
Erreur syntaxe YAML	Faible	Fréquent	Pipeline config (GitLab/GitHub)	< 5 min
Runner/agent offline	Élevée	Moyenne	Runner/Agent GitLab, GitHub, Jenkins	5–30 min
Cache non restauré	Moyen	Occasional	Pipeline caching (GitLab/GitHub)	10–20 min
Artifact manquant	Moyen	Occasion	Pipeline artifact (Azure/CircleCI)	10–20 min
Échec de job (build/test)	Élevée	Fréquent	Pipeline job script	10 min–1h
Tests instables (flaky)	Élevée	Fréquent	Suite de tests CI	1h–3h
Mauvaise gestion des secrets	Élevée	Occasion	Secrets CI/CD (GitHub, GitLab)	15–30 min
Limites de ressources dépassées	Élevée	Occasion	Runners, Agents	30 min–1h

Problème	Gravité	Fréquence	Composant affecté	Temps de résolution
Problème d'intégration (Slack, etc.)	Faible	Occasion	Outils tiers (Slack, etc.)	10–30 min

Déploiement et production (DevOps Cycle)

- **Échecs de build** – Un commit ne compile pas (erreur “*BUILD FAILED*”). *Causes* : code cassé, dépendances manquantes, variables d’environnement non configurées, compatibilité de version. *Dépannage* :
 - Examiner logs de compilation (mvn clean install, npm run build, etc.).
 - Lancer localement pour reproduire le problème.
 - Vérifier la version de l’outil (JDK, Node, Docker) sur l’agent. *Solution* : corriger le code ou les scripts. Ajouter les dépendances dans le pipeline (ex. apt-get ou npm ci). *Prévention* : tests unitaires avant commit, intégration continue (lint, static analysis).
- **Conteneurisation (Docker)**
 - **Build Docker échoue** (erreur “*docker build failed*”). *Causes courantes* : instruction ADD/COPY sur un fichier absent, proxy réseau bloquant, versions incompatibles de dockerfile, espace disque épuisé. *Dépannage* :
 - Lire le message d’erreur : p.ex. failed to resolve source metadata indique un problème de résolution d’URL base.
 - Vérifier que le fichier .dockerignore n’exclut pas de fichiers nécessaires.
 - Tester docker build localement en mode verbeux (--no-cache). *Solution* : corriger le Dockerfile (ordre des instructions comme dans [29]), ajouter --no-cache pour forcer la reconstruction (stackoverflow conseille ce workaround). Vérifier l’accès Internet (proxy) et les droits.
Prévention : maintenir des Dockerfiles simples, réduire les images (multi-stage builds).
 - **Push/Pull d’image échoué** (erreur “*authentication required*” ou “*denied*”). *Causes* : identifiants Docker mal configurés, nom d’image mal formé, permissions IAM manquantes. *Dépannage* :
 - Tenter docker login manuellement pour l’registry (ex. Container Registry, Artifact Registry).

- Vérifier le nom complet de l'image (ex. us-west1-docker.pkg.dev/projet/repo/image:tag).
- Confirmer que le Service Account de build a le rôle artifactregistry.writer ou storage.admin. *Solution* : accorder les droits nécessaires (voir Cloud Build docs). Utiliser gcloud auth configure-docker. *Prévention* : automatiser l'injection des credentials (utiliser Workload Identity pour GKE, tokens CI/CD).

- **Orchestration Kubernetes / GKE**

- **Pod en CrashLoopBackOff / ImagePullBackOff** – Pod ne reste pas en *Running*. *Causes* : image invalide, dépendances manquantes, tag erroné, mount de volume échec, liste blanche (Network Policy) bloquante, permissions de lire une image privée. *Dépannage* :
 - kubectl describe pod <pod> -n <ns> (événements, raison du restart).
 - kubectl logs <pod> -n <ns> pour voir la sortie d'erreur du conteneur.
 - Pour ImagePullBackOff, vérifier kubectl describe : souvent "ErrImagePull" ou "unauthorized" indique un problème d'auth. *Solution* : corriger le tag/image (penser case-sensitivity), appliquer un imagePullSecret si privé. Augmenter les resources.limits si OOM. [37†L333-L342] (doc GKE sur CrashLoopBackOff) recommande d'examiner les logs du conteneur et les événements système. *Prévention* : déployer des image statiques testées, utiliser des probes pour auto-restart (voir plus bas), et mettre en place des redémarrages automatiques (Deployment).
- **Probes Liveness/Readiness en échec** – Le Load Balancer ou Kubernetes signale que les pods ne répondent pas. *Causes* :
 - Path/port mal configuré pour les sondes (ex. readiness probe pointe sur mauvais port).
 - Application n'écoute pas, ou ne répond pas en 200 OK.
 - Firewall GCP bloque l'accès des sondes (health check IP). *Dépannage* :
 - Vérifier les paramètres de livenessProbe/readinessProbe dans le manifest (port, path).

- `kubectl get pods --show-labels` et `kubectl describe service <svc>` pour s'assurer que `targetPort` du Service correspond au `containerPort`. [61†L422-L430] rappelle que le port de `readiness` doit correspondre au port exposé.
- Côté GCP: vérifier les règles de firewall autorisant l'IP range des *health check* (0.0.0.0/0 si doute). *Solution* : ajuster probes (augmenter `initialDelaySeconds` si l'app démarre lentement), corriger path. Créer règle de firewall avec tag réseau `health-check` pour laisser passer le trafic du LB. *Prévention* : mettre des probes soft qui ne tuent pas prématurément, tester localement les endpoints de santé.
- **PodNotReady / Pods peuplan** – Nœud saturé, taints, quotas. *Causes* : `NodeOutOfMemory` (OOM Killed), disques pleins, tolérance/affinité mal définie. *Dépannage* : `kubectl describe node` pour voir usage CPU/mémoire et taints (`MemoryPressure`, `DiskPressure`). Examiner `kubectl get events`. *Solution* : augmenter les ressources du cluster ou adapter les `requests/limits` des pods. Pour OOM, augmenter `limitMemory`, ou optimiser l'application. Pour disk-pressure, ajouter des nœuds ou libérer du disque. *Prévention* : monitoring (Cloud Monitoring) sur usage de nodes, configurer l'**Autoscaler**.
- **Déploiement bloqué (kubectl rollout)** – Deployment en cours de mise à jour mais incapacité à atteindre le bon état, pods restants 0/1 entre parenthèses. *Causes* : image inexploitable ou pods crashant. *Dépannage* : `kubectl rollout status deployment/<dep>` pour voir l'état. *Solution* : revenir en arrière : `kubectl rollout undo deployment/<nom>` (déploie la version précédente). Corriger ensuite l'image/savoir pourquoi. *Diagramme mermaid (rollback)* :

mermaid

Copy

graph TD

A[Déploiement de la version N] --> B[Apply]

B --> C{Checks de santé}

C -->|succès| D[Déploiement finalisé]

C -->|échec| E[Kubectl rollback (undo)]

E --> F[Version N-1 restaurée]

F --> G[Déploiement stable ou notification d'échec]

Prévention : utiliser des stratégies blue/green ou canary (via labels/versioning) pour limiter l'impact.

- **Services et Load Balancers –**

- **Load Balancer GCP:** backends marqués *unhealthy* (ne répondent pas au health check). *Dépannage* : utiliser Connectivity Tests de GCP pour diagnostiquer les règles de firewall et chemins réseau. *Solution* : autoriser explicitement les IP de sondes (voir [60†L126-L133]). Vérifier que la Sonde de santé cible le bon port et protocole (HTTP, SSL). Corriger les CGP URL maps (pas de conflit dans les maps de routage).
- **DNS –** Nom de service non résolu. *Causes* : entrée DNS absente ou TTL non mis à jour, Cloud DNS mal configuré. *Dépannage* : dig +short service.domain.com; gcloud dns managed-zones list. *Solution* : créer/enregistrer l'enregistrement A/CNAME. Mettre à jour TTL faible en période de changement.

- **Réseau (VPC, Firewall, NAT) –**

- **Firewall** : trafic bloqué (ex : pods GKE intranet non joignables, Cloud Run inaccessible). *Dépannage* : contrôler gcloud compute firewall-rules list, examiner les tags réseau attachés aux VM/K8s. *Solution* : ajouter des règles "allow" sur les ports/ports ranges nécessaires (e.g. 80,443, ou le port interne de l'application). Pour l'équilibrage de charge, voir [60†L126-L133] (autoriser health checks).
- **Cloud NAT –** VM ou pods sans IP publique mais sans NAT : accès internet impossible. *Dépannage* :
 - Confirmer que le cluster est en mode *private* (sinon les nœuds ont IP externes).
 - Vérifier règles de route/NAT config (gcloud compute routers nats list).
 - Examiner les logs Cloud NAT (peut être désactivés si log non activé). *Solution* : si cluster privatif, configurer un Cloud NAT correct (associer au sous-réseau). Si non privatif, noter que NAT n'est pas nécessaire pour la sortie (VM a déjà IP externe).
Prévention : bien choisir cluster privatif vs public selon le besoin de contrôler l'accès, activer la collecte de logs NAT pour debug.

- **DNS et routage global/régional –** Comprendre la portée des ressources : certains load balancers (HTTPS/SLL LB) sont globaux, d'autres (Internal LB) sont

régionaux. *Problème* : redirections manquées ou LB ne pointe pas sur la ressource correcte.

Solution : utiliser gcloud compute forwarding-rules list et s'assurer de la région matching. Revoir les configurations multi-région (requiert IPs globales vs IPs régionales).

- **Déploiement blue/green ou canary** – Stratégie de déploiement progressif. *Causes de problème* : label de version incorrect, microservice ancien toujours servi. *Solution* : contrôler les versions via labels (Kubernetes: kubectl label svc, Helm chart versions). Utiliser des outils comme Istio/Kiali pour canary basés sur métriques.
- **Infrastructure as Code (IaC)** – Terraform/Déploiement infra échoue. *Causes* : état corrompu, ACL insuffisantes, quotas (CPU, API). *Dépannage* :
 - terraform plan pour localiser le changement.
 - terraform refresh pour mettre à jour l'état.
 - Vérifier les logs Cloud Logging sur les API (Compute, GKE) pour repérer les erreurs d'API. *Solution* : corriger l'état (via terraform import si objet manquant, ou taint pour forcer recreate). Ajuster permissions du SA (ex. roles/compute.admin). *Prévention* : verrouiller l'état (backend remote), suivre les meilleures pratiques (Terraform Cloud, revue PR de .tf).
- **Observabilité & monitoring** –
 - *Logs & traces* : centraliser les logs d'erreurs (Stackdriver/Cloud Logging, ou autres ELK). *Problème* : logs manquants ou non-indexés. *Solution* : configurer Cloud Logging agents ou stack de logs, consigner les erreurs côté applicatif (console.error, stderr).
 - *Métriques & alertes* : absence d'alerte sur incidents. *Solution* : définir des alertes Cloud Monitoring sur les métriques clés (CPU, latence, erreurs 5xx). Utiliser des dashboards (GCP Monitoring).
 - *Debug* : instruire l'équipe à utiliser kubectl logs --since=1h ou gcloud logging read "severity>=ERROR" pour requêter les erreurs récentes.
- **Sécurité & IAM** –
 - *Permissions* : erreurs "Permission denied" courantes si un compte n'a pas de rôle adéquat (exemple Cloud Build ActAs, Cloud Run Developer pour déployer Cloud Functions). *Solution* : donner les rôles manquants (ex. roles/cloudbuild.builds.editor, roles/iam.serviceAccountUser).

- *Service Accounts* : le SA utilisé par une VM/GKE ne peut pas accéder à un autre service (ex. Composer, Pub/Sub) – vérifier les scopes OAuth attachés à la VM.
- *Mauvaises pratiques* : ne jamais exécuter de commandes gcloud en local avec un compte persistants sans MFA, préférer Workload Identity (GKE) ou IAM sur les répertoires.

Déploiement sur Google Cloud Platform (GCP)

- **Google Kubernetes Engine (GKE) –**

- *Création de cluster* échoue (quota CPU, régions indisponibles) : contrôler quotas via `gcloud compute project-info describe`. Activer les APIs (GKE, Compute).
- *Clusters autopvisionnés* : vérifier que le projet n'est pas surchargé (limite 5 clusters standard par zone).
- *Pods CrashLoopBackOff* : se référer à la doc GKE pour diagnostiquer (consulter `kubectl get events`, logs de conteneur).
- *Autoscaling* : si pods ne montent pas, regarder `kubectl get hpa` ou `gcloud container clusters describe`.
- *Mises à jour de version* : pods crash après upgrade vers v1.25+? Voir dépréciations d'API.
- *Accès kubectl* : vérifier que les bons IAM (kubernetes-engine.viewer/clusterAdmin) sont attribués. Utiliser `gcloud container clusters get-credentials`.
- *Nodes NotReady* : résoudre ressources (CPU, Memory) ou problèmes de réseau interne.
- *Ingress et LoadBalancer* : s'assurer que la spécification du Service de type LoadBalancer a les bons ports, labels et que le VPC/filtrage autorise le trafic entrant.

- **Cloud Run –**

- *Déploiement échoue* : erreurs 403/404 sont souvent dues au projet désactivé ou manquant d'API. Par ex. l'erreur **SYSTEM_DISABLED** indique un projet désactivé. *Solution* : réactiver le projet (Facturation, API).
- *Limites de conteneurs* : mémoire ou CPU trop faibles entraînent plantage (utiliser le `--memory` et `--timeout` corrects).

- *Cold starts/latence* : voir métriques de latence (Cloud Monitoring), augmenter min-instances ou ajuster concurrency.
- *Appels externes* : Cloud Run isolé VPC (Serverless VPC connector) mal configuré provoque timeouts.
- *Authentication* : Cloud Run peut nécessiter des configurations IAM strictes (Service Account attaché, OIDC).
- **Cloud Build –**
 - *Permissions manquantes* :
 - *Logs Cloud Storage* : erreur *AccessDenied...storage.objects.get* signifie qu'il faut donner le rôle *Project Viewer* à l'utilisateur lançant le build.
 - *ServiceAccounts.actAs* : si on déploie via Cloud Build sur Cloud Run/Functions/AppEngine, il faut que le SA de Cloud Build puisse *impersonate* le SA cible (erreur *iam.serviceAccounts.actAs*).
 - *Déclencheur* : *cloudbuild.builds.create* denied requiert le rôle *Cloud Build Service Account* pour le SA configuré. *Solution* : suivre les conseils du troubleshooting de Cloud Build, attribuer correctement les rôles.
 - *Échecs génériques* : config du fichier *cloudbuild.yaml* invalide, quotas de builder (vCPU/mémoire) épuisés. Augmenter CPU avec *--builder-cores* ou réutiliser *--no-cache* si images en cache posent problème.
- **Artifact Registry / Container Registry –**
 - *403 permission* en docker push/pull : souvent l'utilisateur n'a pas *artifactregistry.writer*. Pour Cloud Build, voir guide OneUptime.
 - *Nom d'image incorrect* : avec Artifact Registry, toujours inclure le nom de repository (ex. *region-docker.pkg.dev/projet/repo/image*).
 - *Migration gcr.io→pkg.dev* : si on migre, mettre à jour les triggers CI/CD.
 - *VPC SC* : en réseau isolé, tirer de Artifact Registry peut échouer (ex. pulling l'image du sidecar Prometheus). Il faut autoriser le projet *cloud-ops-agents-artifacts* dans les règles VPC SC.
- **Compute Engine –**
 - *Instances qui ne démarrent pas* : voir logs sous */var/log/syslog* (Debian/Ubuntu) ou */var/log/messages*. Problème

de **startup script** mal formulé : vérifier les métadonnées de l'instance (gcloud compute instances describe <vm>).

- *SSH inaccessible* :
 - Utiliser gcloud compute ssh <vm> pour forcer l'ajout de clés.
 - Voir le doc Compute sur SSH Troubleshoot. Erreur "*Permission denied (publickey)*" indique souvent une clé manquante ou inutilisable ; résoudre via métadonnées projet.
- *Firewall / Règles réseau* : VM hors du sous-réseau ou tag réseau incorrect, induit impossibilité de ping/connexion. *Solution* : gcloud compute firewall-rules create allow-http --allow tcp:80. Vérifier les réseaux (VPC regional vs global).
- *Quota/regional vs zonal* : tenter de créer une VM dans une zone sans quota épuisé ou utiliser une autre région.
- *Billing* : si facturation désactivée, aucune VM ne démarre – consulter le message en console GCP ou CLI (gcloud compute instances list renvoie des erreurs 403).
- **Réseau et Load Balancing** –
 - **Règles de pare-feu manquantes** (Cloud Run, GKE, Compute) : Par exemple, pour GKE Ingress, vérifier les sources IP santé (health check), et ajouter un firewall *Ingress* permettant 0.0.0.0/0 sur port défini. Pour Cloud Run, si interne, configurer l'ingress interne (VPC egress).
 - **Cloud NAT** – voir GKE: si cluster *privé* et pas de NAT, les pods *Cannot reach internet*. Configurer Cloud NAT ou activer Public IP sur nœuds. [66†L158-L167] explique que sans NAT, les nœuds d'un cluster non-private disposent d'IPs externes par défaut.
 - **Load Balancer (LB)** :
 - Backend en *Draining* (cannot add backends) ou *Timeout* : vérifier que les instances sont dans la même région/rézo (sous-réseau) que le LB.
 - HTTPS LB: erreurs de certificat SSL (404) si la ressource backend ne correspond pas.
 - Régional vs global: utiliser --global pour IPs LB global, --region pour régional.

- **DNS** : créer et vérifier les enregistrements DNS via **Cloud DNS**. TTL élevé peut retarder les changements d'adresse IP. Utiliser dig pour debug.
- **Mises à jour Blue/Green** : déployer les deux environnements, basculer le trafic. *Problème* : le *switch* n'a pas été fait. *Commande* : avec GCP, mettre à jour la cible du DNS ou du forwarding rule (gcloud, console).
- **Canary** : monitorer la nouvelle version à faible pourcentage de trafic (via GKE Istio ou Cloud Run Traffic Splitting).
- **IAM et sécurité GCP** –
 - **Rôles inadaptés** : erreurs Denied (ex. BigQuery, Pub/Sub) lors de test de bout en bout. *Solution* : attribuer les rôles gérés ou créer un custom role minimal. Toujours privilégier le principe du moindre privilège.
 - **Service Accounts** : ne pas confondre SA utilisateur avec SA de VM. Exemple : pour que Cloud Build écrive dans Container Registry, le SA de Cloud Build doit avoir roles/storage.admin sur le projet (pour GCR legacy).
 - **Metadata Server et scopes** : sur Compute, ne pas oublier que les VM doivent avoir les scopes OAuth adéquats pour accéder aux APIs (ex. default [compute, cloud-platform]).

Sample commands diagnostiques :

bash

Copy

gcloud compute instances list

gcloud container clusters list

kubectl get pods --all-namespaces

kubectl describe pod <pod> -n <ns>

kubectl logs <pod> -n <ns>

gcloud build operations list

docker pull gcr.io/\$PROJECT/myapp:tag